

Разработка драйверов в Linux, 3

Владислав Богданов

2026-06-16

Оглавление

1. Символьное устройство	1
2. Номера устройств	1
3. Как выбрать номера	2
4. Работа с символьным устройством	2
5. Как это всё связывается воедино (Архитектура VFS)	3

1. Символьное устройство

Три Unix, Linux основных типа устройств * символьные - устройство к которому можно обращаться и интерпретировать как поток байтов, как к файлу, поэтому для работы используются файлы символьных устройств. Это одна из разновидностей 7 типов файлов(нодов) в Linux, помечаются в директории /dev буквой c.

Виды файлов в Linux (7 типов нодов):

- * обычные файлы
- * директории
- * каналы
- * сокеты
- * символьные ссылки
- * символьные устройства (буква C)
- * блочные устройства (буква B)

Взаимодействи с помощью - потоки ввода-вывода - memory paging и др

- блочные - дисковые устройства, может содержать файловую систему, взаимодействие выполняется с помощью записи/чтения блоков, может быть виртуальным.
- сетевые интерфейсы, может быть просто логикой программой, не обязательно устройство.

2. Номера устройств

Два числа 5, 0 и тд это старший и младший номера устройств (major, minor). Пользовательское по работает в пространстве пользователя и имена файлов устройств с реализацией устройства в виде модулей связывают специальным образом.

Старший номер можно получить через /proc/devices, где он сопоставлен с названием модуля. Например, 1 — это mem, который отвечает за null.

```
crw-rw-rw- 1 root tty 5, 0 Jul 7 12:00 tty
```

```
crw----- 1 root tty 4, 0 Jun 25 10:17 tty0
```

```
crw----- 1 root tty 4, 1 Jun 25 10:17 tty1
```

Старший номер определяет драйвер (модуль) обеспечивающий работу устройства.

Младший номер устройства позволяет точно определить о каком устройстве идет речь(конкретный вид).

Уточнение:

- Исторически (в очень старых Unix) номера были 8-битными.
- В современном Linux тип для номера устройства — это dev_t (обычно 32 бита).

- Старший номер (Major): выделяется 12 бит (максимум 4095).
- Младший номер (Minor): выделяется 20 бит (максимум 1 048 575).
- Для работы с ними в ядре есть макросы: MAJOR(dev), MINOR(dev) и MKDEV(major, minor).

Младшие номера позволяют, например, различать десятки терминалов tty. Старший номер можно: назначить вручную (с риском конфликта); или попросить систему выделить свободный (лучше). Файлы устройств (в /dev) можно создавать вручную или программно, а номер передавать как параметр при загрузке модуля.

3. Как выбрать номера

1. Статическое выделение (register_chrdev_region): ты сам жестко задаешь номер. Плохо, потому что может возникнуть конфликт с другим драйвером. Используется только для очень специфичных случаев.
2. Динамическое выделение (alloc_chrdev_region): ядро само находит свободный старший номер. Это правильный путь.
3. Создание файла в /dev:
 - Ручное (устаревшее для продакшена): `sudo mknod /dev/mydev c <major> <minor>`.
 - Автоматическое (современное): Драйвер в ядре не должен сам создавать файлы в /dev. Этим занимается подсистема udev (или mdev во встроенных системах). Драйвер только сообщает ядру: "Я создал устройство с таким-то номером и классом", а udev в user-space ловит событие (uevent) и сам создает красивый файл в /dev.

4. Работа с символьным устройством

`struct file_operations` - описание того как модуль символьного устройство обрабатывает запросы пользователя. Описывает реализации (каждый элемент — указатель на функцию):

- `open()` — открытие;
- `release()` — закрытие (аналог `close`);
- `read()` — пользователь читает → модуль пишет;
- `write()` — пользователь пишет → модуль читает.

Там же описан владелец (owner) Помимо этого может быть реализация:

- `lseek()`
- асинхронное чтение и тд

Поэтому для надежности структуру `file_operations` заполняют нулями.

Эту структуру нужно объявлять как `const`. Зачем? Чтобы она попала в read-only секцию памяти ядра. Это защищает таблицу указателей от случайной перезаписи (баги в ядре) и экономит RAM. Поле `owner` всегда должно быть равно `THIS_MODULE`. Это нужно ядру, чтобы знать, какому модулю принадлежит функция, и не дать пользователю выгрузить модуль

(rmmod), пока файл открыт.

struct file - определяет файл как элемент файловой системы в Linux на уровне ядра, не имеет никакого отношения к обычным интовым дескрипторам. Эта структура вообще в ядре не определена и не имеет смысла. Ни когда не появляется в пользовательских программах. Представляет собой описание открытого файла с которым ядро работает, создается заполняется ядром, при открытии файла, любого и содержит всю необходимую служебную информацию о файле как об элементе файловой системы. То есть туда например входит как раз структура `file_operations`, что бы было понятно как работать с данным файлом, там содержатся флаги определяющие доступ и порядок работы с файлом и тд.

struct inode - можно получить данные о непосредственно нашем файле в соответствии с его типом, содержит описание самих символьных устройств.

struct cdev

1. Алгоритм инициализации символьного устройства в ядре выглядит так:
2. Динамически просим номера: `alloc_chrdev_region(&dev_num, ...)`
3. Создаем и инициализируем cdev: `cdev_init(&my_cdev, &my_fops);` (связываем операции с устройством).
4. Добавляем устройство в систему: `cdev_add(&my_cdev, dev_num, count);` (Опционально, но желательно) Создаем класс и устройство, чтобы udev сам создал файл в `/dev`: `class_create(), device_create()`.

5. Как это всё связывается воедино (Архитектура VFS)

Давай проследим путь, когда пользователь делает `read(fd, buf, 10)`, чтобы ты понял, как эти структуры работают вместе.

1. User-space: Программа вызывает `read(3, buf, 10)`.
2. Системный вызов: Ядро берет `fd = 3`, смотрит в таблицу процесса и находит `struct file`.
3. VFS (Virtual File System): Ядро смотрит внутрь `struct file` и находит указатель на `struct file_operations` (`file → f_op`).
4. Диспетчеризация: Ядро проверяет, задан ли там указатель `f_op → read`. Если да — вызывает его, передавая туда буфер, размер и саму `struct file`.
5. Твой драйвер: Твоя функция `my_read(struct file *filp, ...)` выполняется.
6. Поиск устройства: Внутри `my_read` тебе нужно понять, к какому именно "железу" или буферу обращаться. Ты берешь `filp` (это `struct file`), через неё получаешь доступ к `struct inode` (`file_inode(filp)`), а из `inode` достаешь `minor` номер, чтобы понять, с каким именно конкретным устройством из твоего драйвера работать.

Реализовать модуль в модуль передается диапазон для генерации случайных значений при чем есть параметры по умолчанию, так же в модуль передается массив чисел через параметры модуля.

проверка на превышение размера массива,

можно без массива

модуль должен обновить содержимое элементов данного массива умножив на случайное число

посчитать сумму чисел и вывести в лог.

во время работы модуля можно читать и изменять параметры если во время работы программы параметр модуля соответствующий диапазону случайных чисел меняется случайные числа должны быть сгенерированы заново